Remote Job Finder for Claude — Product Spec

Oct 20, 2026 · @Abdullah Yaqub

Overview

A remote MCP connector that turns a user's own Claude into a job-hunting agent: it finds real remote jobs they are eligible for, flags ghost jobs, and builds a tailored CV per job. Users bring their own Claude subscription, so the product carries no LLM cost.

- **Who it's for:** existing Claude Pro/Max/Team users looking for remote roles, with an early focus on candidates outside the US (Pakistan, South Asia, MENA). Most remote listings exclude them, and no tool filters for that well.
- **What we own:** job data (ingestion, dedupe, repost history), ghost signals, user profiles, CV compile, and the application tracker.
- **What the user's Claude does:** all reasoning. That covers fit scoring, the ghost-risk verdict, CV writing, and screening answers, guided by our plugin's prompts.
- **Moat:** months of repost history per company (ghost detection gets better with time) plus location-eligibility data that general job boards don't expose.
- **Not in scope:** auto-applying. Users submit applications themselves, which keeps accounts safe and quality high.

User journey

The user goes from signup to a submitted application inside their own Claude chat. They never open our website except to log in and pay.

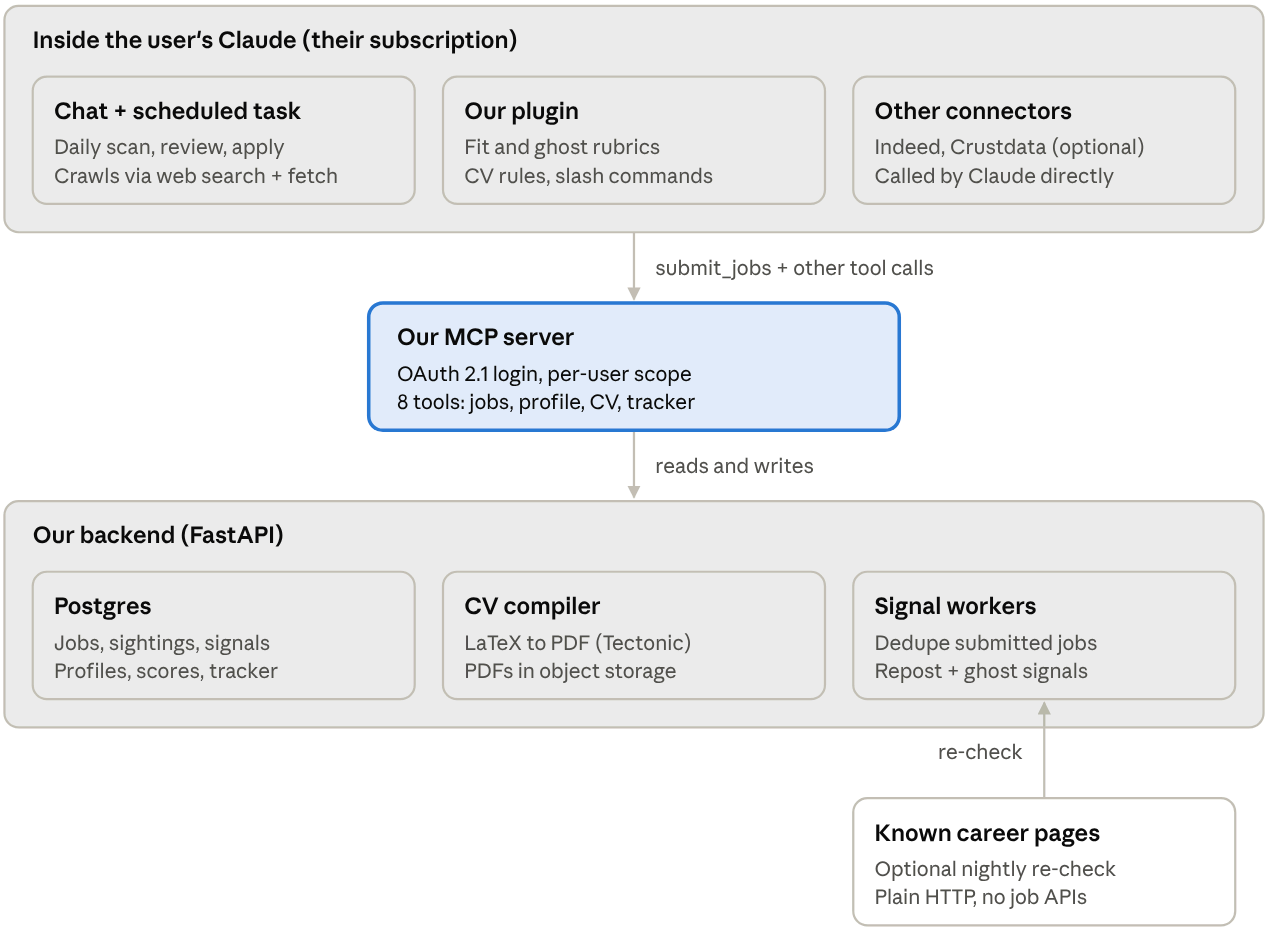
1. **Install:** the user adds the connector in Claude (custom connector URL until we're listed in the directory), then logs in through our OAuth page.
2. **Onboard:** the user uploads their CV (PDF or LaTeX) and Claude turns it into a structured **Master Profile** through `update_profile`. The user then sets preferences: target titles, salary floor, US-overlap hours, full-time or contract, and countries they can legally work from.

3. **Daily discovery:** a scheduled task in the user's Claude runs every morning. It calls `search_jobs`, pulls jobs already in our store, crawls the web for new ones, and sends them back through submit_jobs, and scores fit. It then posts a shortlist ranked Match → Stretch → Reach, with high ghost-risk jobs hidden.
4. **Review:** the user opens a job, and Claude explains the fit, the gaps, and the ghost-risk reasons.
5. **Application kit:** the user says "apply to this one." Claude calls `generate_cv`, which returns a tailored PDF plus a gap report, a cover note, screening answers, and a recruiter DM.
6. **Submit:** the user applies on the company site and marks it applied through `track_application`.
7. **Follow-up:** Claude reminds the user of follow-ups after 7 days, and the tracker records outcomes. Those outcomes feed back into scoring later.

System architecture

There are three layers. The user's Claude reasons, our MCP server is the only door into our system, and our backend owns the data and does the heavy lifting without any LLM.

The user's Claude does the thinking; our backend owns the data



architecture · 3 layers

Other connectors such as Indeed or Crustdata talk to the user's Claude, never to our server, so our own ingestion has to stand on its own. Workers write into Postgres, and the MCP tools read from it.

Job discovery: Claude crawls

The user's Claude does the crawling. It searches the web and opens career pages with its own web search and fetch tools, then sends what it found to our server with `submit_jobs`. No paid job APIs are involved.

Where Claude looks	How	Notes
Search engines (e.g. `site:boards.greenhouse.io backend remote`)	Web search	Finds fresh postings across ATS boards and job sites
Company career pages and ATS boards (Greenhouse, Lever, Ashby, Workable)	Web fetch	Public pages; the strongest anti-ghost signal
Remote boards (Remotive, RemoteOK, Himalayas, We Work Remotely)	Web fetch of listing pages	Remote-first, often with location tags
LinkedIn and Indeed listings	Web search results only	Usually behind login walls. Claude uses the result to find the same job on the company's own site

Pipeline steps:

1. **Plan:** the plugin's `daily-scan` skill builds search queries from the profile (target titles × sites × "remote").
2. **Crawl:** Claude runs the searches and opens the most promising pages, within a fixed per-run budget of page fetches. The budget keeps a scan from using up the user's plan limits.
3. **Extract:** for each page, Claude pulls the title, company, location text, salary, posted date, apply URL, and JD text.
4. **Submit:** `submit_jobs` sends the batch to our server. The server normalizes it, dedupes on company + title + a JD hash, and records a sighting.
5. **Shared history:** every user's crawl adds sightings, so repost and age signals build up across all users. In effect, the users are the crawler.
6. **Re-check (optional):** a lightweight nightly server job re-opens known career-page URLs over plain HTTP to mark closed jobs and catch reposts.

Tradeoffs: crawling runs inside the user's plan limits, takes a few minutes per scan, and coverage depends on what search surfaces. Respect robots.txt and site terms, and do no logged-in scraping of LinkedIn.

Eligibility and ghost-job scoring

Two checks run before a job reaches the user: an eligibility filter that removes jobs they can't legally take, and a ghost-risk score that ranks how likely a posting is to be real.

Eligibility (hard filter). Each job's location text is classified into one of three groups:

- **Worldwide**
- **Region-open:** EMEA, APAC, or a specific list of countries
- **Restricted:** US-only, EU work permit required, and similar

A job shows only if the user's country fits. Required timezone overlap is compared against the user's overlap hours. The backend uses rules plus keyword matching first, and the user's Claude resolves ambiguous cases.

Ghost risk (score 0–100, shown as Low / Medium / High):

Signal	Computed by	Points added to risk
Not found on the company's own ATS or careers page	Backend	+35
Reposted 2+ times in 90 days (same company + title)	Backend, from sightings	+25
Posted more than 45 days ago	Backend	+15
Company headcount flat or falling, or recent layoffs	User's Claude via news search	+15
Vague JD: no team, no responsibilities, "talent pool" wording	User's Claude via plugin rubric	+10
Salary range listed	Backend	−10
Named hiring manager or recruiter	User's Claude	−5

Bands: 0–29 Low, 30–59 Medium, 60+ High. High-risk jobs are hidden by default but kept in a list, with reasons, that the user can open. The weights are a starting point and get tuned against application outcomes (interview rate per band).

Fit scoring

Fit is judged on the requirements a job actually lists, not its "years of experience" number, so a user can reach jobs at or above their level. The user's Claude does the scoring, following the plugin rubric, and saves the result with `save_job_score`.

Band	Rule of thumb	Recommended action
Match	Meets all must-haves; years asked are within ~1 year of theirs	Apply now
Stretch	Missing 1–2 must-haves that are transferable, or asks 2–3 more years	Apply with a gap-aware CV
Reach	Missing core stack or domain, or asks 4+ more years	Apply only with a referral

Each score includes three things:

- the must-haves met, with evidence from the profile
- the gaps
- a one-line verdict

Scores are cached per (profile version, job), so a job is scored only once per profile change.

CV and application kit

The user's Claude writes the tailored content, and our server compiles it into a PDF. Every claim in the CV must trace back to the Master Profile.

Master Profile: structured JSON covering roles, projects, bullets with metrics, skills with evidence, education, and links. It is versioned, edited only through `update_profile`, and the user approves every change.

Generation flow:

1. Claude reads the job and the Master Profile.
2. It selects the most relevant bullets, reorders sections, and rewrites bullets in the job's vocabulary.
3. It fills one of our LaTeX templates. Users can upload their own template.

4. `generate_cv` compiles the template with Tectonic and returns a PDF link. The `.tex` source is kept for edits.

No-fabrication rule. Claude may rephrase, reorder, and omit, but it may not add a skill, employer, metric, or year that isn't in the profile. The server checks this too: every skill named in the CV must exist in the profile, or the compile is rejected.

Kit contents:

- tailored CV (PDF + .tex)
- gap report: what the job asks for that the user lacks, and how to address it honestly
- cover note (about 150 words)
- answers to the job's screening questions
- a recruiter / hiring-manager DM

MCP tools and plugin

Nine tools cover the MVP. The plugin ships the prompts, so every user's Claude scores and writes the same way.

Tool	Input	Returns
`get_profile`	—	Master Profile + preferences, plus search queries for the crawl
`update_profile`	profile patch or uploaded CV text	new version, diff for user approval
`submit_jobs`	jobs Claude extracted while crawling	per job: new or known, sighting count, ghost signals
`search_jobs`	filters (titles, since, band), limit	eligible jobs already in our store, unscored first
`get_job`	job_id	full JD, sightings history, signals
`save_job_score`	job_id, band, ghost verdict, reasons	saved score
`generate_cv`	job_id, LaTeX body	PDF link, .tex, validation result
`track_application`	job_id, status, notes	updated tracker row

Tool	Input	Returns
`list_applications`	status filter	tracker with follow-up dates

Plugin / skill bundle:

- `daily-scan` skill: the steps for the morning scheduled task (crawl → submit_jobs → score → shortlist message)
- `fit-rubric` and `ghost-rubric`: the scoring rules from the sections above
- `cv-writer`: tailoring rules, the no-fabrication rule, gap-report format
- `/jobs`, `/apply <job>`, `/tracker` slash commands as shortcuts

Transport: Streamable HTTP, with OAuth 2.1 for per-user auth. Tool descriptions stay short and explicit, because they decide when Claude calls them.

Data model

Postgres holds the data. Jobs are shared across users, and everything personal is scoped by `user_id`.

Table	Key columns	Notes
`users`	id, email, plan, created_at	From OAuth signup
`profiles`	id, user_id, version, data (jsonb), preferences (jsonb)	Versioned Master Profile
`companies`	id, name, domain, ats_type, ats_slug, headcount_trend	Seed list + enrichment
`jobs`	id, company_id, title, title_norm, location_text, eligibility, remote_type, salary_min/max, posted_at, apply_url, jd_text, jd_hash, status	One row per unique job
`job_sightings`	id, job_id, source, seen_at, source_url	Powers repost and age signals
`ghost_signals`	job_id, on_ats, repost_count_90d, age_days, salary_listed, risk_score	Recomputed after each fetch

`job_scores`	user_id, job_id, profile_version, band, ghost_verdict, reasons	Written by the user's Claude
`cvs`	id, user_id, job_id, tex, pdf_url, created_at	Generated kits
`applications`	id, user_id, job_id, status, applied_at, follow_up_at, outcome	Tracker; outcomes tune the rubrics

Costs and monetization

Running costs are only hosting and data APIs, because users' Claude subscriptions cover all LLM usage.

- **Hosting:** one small VM or container for the API + workers, managed Postgres, and object storage for PDFs.
- **Crawling:** zero for us. It runs on the user's Claude web search and fetch, within their plan limits.
- **Nightly re-check (optional):** plain HTTP requests to known career pages, which costs almost nothing.
- **Enrichment (optional):** none needed; Claude checks layoffs and funding news itself, cached once per company per week.

Pricing (to validate):

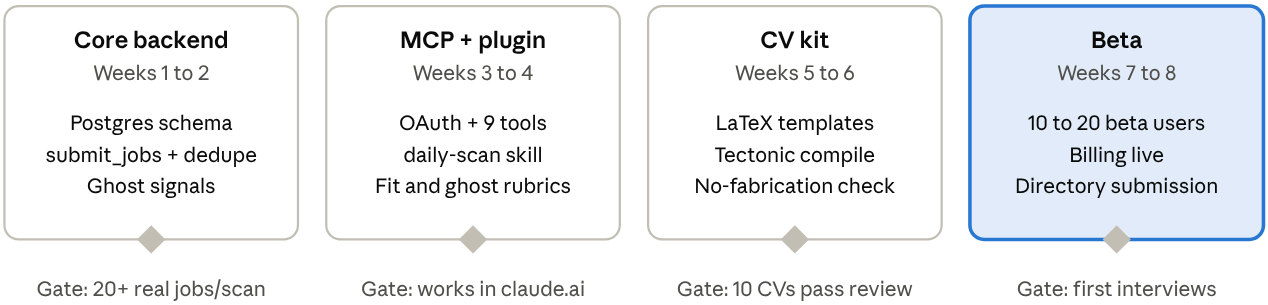
- Free: daily shortlist + 3 CVs a month
- Paid: unlimited CVs, full ghost history, follow-up reminders

Billing runs on our website through Stripe or Lemon Squeezy, linked to the OAuth account. The exact price is an open question.

MVP roadmap

The proposed plan is four 2-week phases, each ending in a gate that must pass before the next phase starts. Abdullah is the first user from week 3, so his own job search is the test bed.

The MVP reaches beta users in about 8 weeks



MVP roadmap · 4 phases, 4 gates

The week counts assume part-time solo work and are estimates to revisit after phase 1.

Risks and open questions

Risk	Mitigation
Market limited to Claude subscribers	Accept for MVP; a standalone web app on the same backend later
Each user's Claude behaves differently	Plugin rubrics + strict tool descriptions + server-side validation
Directory review delays public listing	Launch as a custom connector URL; apply to the directory in parallel
A job board changes its API or terms	Crawl many sources; prefer company career pages; respect robots.txt; no logged-in scraping
Anthropic or a job board ships something similar	Niche on non-US eligibility + ghost history data
Profile and CV data is sensitive	Encryption at rest, per-user scoping, delete-my-data endpoint

Open questions:

- [ ] Price point for the paid tier
- [ ] Which 200–500 companies go in the first ATS seed list
- [ ] Include contract / hourly platforms (Deel, Turing) at launch or later?

- [ ] Keep user-uploaded LaTeX templates, or only our own templates in v1?